Assignment 6-1: Complete Study Guide

Policy Evaluation with TD(0)

Temporal Difference Learning | Cliff Walking Environment

PART 1: The Big Picture — What Is This Lab About?

This lab is about teaching an AI agent to PREDICT how good each position (state) is, given a fixed policy (strategy). This is called Policy Evaluation.

The core question: If I follow a specific strategy forever, how much total reward will I expect to get from each state?

To answer this, the lab uses a method called TD(0) — Temporal Difference learning with a one-step lookahead. This is a foundational model-free reinforcement learning algorithm that learns directly from experience, without needing a model of the environment.

Key Concepts You Must Know

Policy Evaluation (Prediction): Given a policy π (a rule for choosing actions), estimate the value function V(s) for each state s. V(s) = expected sum of future discounted rewards starting from state s and following policy π forever.

Model-Free Learning: The agent does NOT know transition probabilities P(s'|s,a) or rewards R(s,a). It learns purely from experience by interacting with the environment.

Online Learning: Updates happen during the episode, not just at the end (unlike Monte Carlo methods which wait until episode completion).

Bootstrapping: TD methods update estimates using OTHER estimates (V(s') is used to update V(s)), rather than waiting for the actual return. This is the key difference from Monte Carlo.

PART 2: The Cliff Walking Environment

Environment Overview

The Cliff Walking environment is a classic gridworld problem. Here's the setup:

- Grid size: 4 rows × 12 columns (default) = 48 states
- Start (S): Bottom-left corner = position (3, 0) = state 36
- Goal (G): Bottom-right corner = position (3, 11) = state 47
- Cliff: Bottom row between S and G = positions (3,1) through (3,10) = states 37–46
- Actions: UP (0), LEFT (1), DOWN (2), RIGHT (3)

Reward Structure

The environment is designed to penalize the agent for every step it takes:

- Normal step: -1 reward (encourages finding shorter paths)
- Fall off cliff: -100 reward + reset to start (big penalty, but episode continues)
- Reach goal: -1 reward + episode ENDS (terminal state)

KEY INSIGHT: The cliff penalty (-100) is much larger than normal steps (-1), creating a trade-off between safe paths (avoiding the cliff) and short paths (near the cliff).

State Representation

States are represented as single integers using row-major order:

```
state = row × grid_width + column
```

Example with 4×12 grid:

- (0,0) = 0×12 + 0 = state 0 (top-left)
- (0,11) = 0×12 + 11 = state 11 (top-right)
- (3,0) = 3×12 + 0 = state 36 (START — bottom-left)
- (3,11) = 3×12 + 11 = state 47 (GOAL — bottom-right)
- (3,1) through (3,10) = states 37–46 (CLIFF)

Movement Rules

Actions change coordinates as follows:

- UP (0): row decreases by 1 → x = x - 1
- LEFT (1): column decreases by 1 → y = y - 1
- DOWN (2): row increases by 1 → x = x + 1
- RIGHT (3): column increases by 1 → y = y + 1

Boundary rule: If movement would go outside the grid, the agent STAYS in place (no movement).

Episode Logic

1. Episode starts: agent placed at start_loc = (3, 0)
2. Agent takes actions step by step
3. If agent reaches cliff: gets -100, resets to start, episode CONTINUES
4. If agent reaches goal: gets -1, episode ENDS (terminal=True)

Important: The episode only ends when the agent reaches the GOAL. Falling off the cliff does NOT end the episode — it just resets position and gives a large penalty.

PART 3: Understanding Each Function

3.1 env_init() — Initialize the Environment

Called ONCE when the environment object is created. Sets up all configuration:

```
self.grid_h = env_info.get('grid_height', 4)   # number of rows
self.grid_w = env_info.get('grid_width', 12)   # number of columns
self.start_loc = (self.grid_h - 1, 0)          # bottom-left
self.goal_loc = (self.grid_h - 1, self.grid_w - 1)  # bottom-right
self.cliff = [(self.grid_h-1, i) for i in range(1, self.grid_w-1)]
```

The cliff is a list of (row, col) tuples in the bottom row between start and goal.

Why row_h - 1? Because rows are 0-indexed. For a 4-row grid: rows 0,1,2,3. Bottom row = row 3 = grid_h - 1.

3.2 state() — Convert Coordinates to State Index

Converts a 2D (row, col) location to a single integer state ID:

```
return loc[0] * self.grid_w + loc[1]
```

This is row-major flattening. For a 4×12 grid:

- Row 0 → states 0–11
- Row 1 → states 12–23
- Row 2 → states 24–35
- Row 3 → states 36–47

The agent only sees state numbers, not (row, col) coordinates. This keeps the agent general — it doesn't need to know the grid shape.

3.3 env_start() — Begin a New Episode

Called at the start of EVERY episode. Resets the agent to the starting position:

```
reward = 0                          # no reward before first action
self.agent_loc = self.start_loc     # place agent at (3,0)
state = self.state(self.agent_loc) # convert to integer (= 36)
termination = False                 # episode not over yet
self.reward_state_term = (0, state, termination)
return state                        # return initial state to agent
```

The function returns the initial state so the agent can choose its first action.

3.4 env_step() — Execute One Action

Called at EVERY step. Given an action, moves the agent and returns (reward, new_state, terminal):

Step 1: Get current position
```
x, y = self.agent_loc
```

Step 2: Apply action
```
if action == 0:    x = x - 1    # UP
if action == 1:    y = y - 1    # LEFT
if action == 2:    x = x + 1    # DOWN
if action == 3:    y = y + 1    # RIGHT
```

Step 3: Check boundaries — if out of bounds, stay put
```
if not isInBounds(x, y, self.grid_w, self.grid_h):
    x, y = self.agent_loc
```

Step 4: Check goal and cliff
```
if agent is at goal:  terminal = True, reward = -1
if agent is at cliff: reward = -100, reset to start
```

The isInBounds() function checks: 0 <= x <= height-1 AND 0 <= y <= width-1

3.5 env_cleanup() — Reset After Episode

Simple cleanup — just resets agent position to start:
```
self.agent_loc = self.start_loc
```

This is called after each episode ends to prepare for the next one.

PART 4: The TD(0) Agent — How It Learns

4.1 agent_init() — Set Up the Agent

Called once. Initializes all agent parameters:

```
self.rand_generator = np.random.RandomState(seed)
self.policy = agent_info.get('policy')    # shape: (num_states, num_actions)
self.discount = agent_info.get('discount') # gamma γ
self.step_size = agent_info.get('step_size') # alpha α
self.values = np.zeros(self.policy.shape[0]) # V(s) = 0 for all s
```

The policy is a 2D array where policy[s] is a probability distribution over actions. For a 48-state, 4-action environment: policy has shape (48, 4).

The value function starts at zero everywhere. TD learning will gradually refine these estimates based on experience.

4.2 agent_start() — First Action in Episode

Called once per episode with the initial state:

```
action = rand_generator.choice(range(4), p=self.policy[state])
self.last_state = state
return action
```

The agent samples an action from the policy distribution at the current state, stores the state for future updates, and returns the action to take.

4.3 agent_step() — The Core TD(0) Update

This is the heart of the algorithm. Called at every step during an episode:

The TD(0) Update Rule

```
V(s) ← V(s) + α [r + γ·V(s') - V(s)]
```

Where:

- V(s) = current value estimate of the previous state
- α (alpha) = step size / learning rate (controls how fast we update)
- r = reward received for the transition
- γ (gamma) = discount factor (how much we value future rewards)
- V(s') = current value estimate of the new state (BOOTSTRAP)
- r + γ·V(s') - V(s) = the TD Error (the surprise/correction signal)

```
In code:
self.values[self.last_state] += self.step_size * (reward + self.discount *
self.values[state] - self.values[self.last_state])
```

Then pick next action and update last_state:

```
action = rand_generator.choice(range(4), p=self.policy[state])
self.last_state = state
return action
```

Understanding the TD Error

The TD Error = r + γ·V(s') - V(s)

- If TD Error > 0: V(s') (estimated future value) is higher than expected — we underestimated V(s), so INCREASE it
- If TD Error < 0: V(s') is lower than expected — we overestimated V(s), so DECREASE it
- If TD Error = 0: our estimate is exactly consistent — no update needed

The TD error measures how 'surprised' the agent is. It drives learning toward consistent value estimates.

Why TD(0) is Powerful

- Updates at EVERY step — no waiting until episode end
- Works in continuing tasks (no natural episode endings)
- Bootstraps — uses V(s') to estimate V(s), propagating information backward
- Lower variance than Monte Carlo (doesn't depend on the full return)
- Higher bias than Monte Carlo (because V(s') is itself an estimate)

4.4 agent_end() — Terminal State Update

Called when the episode ENDS (agent reaches goal). There is no next state, so V(s') = 0:

```
self.values[self.last_state] += self.step_size * (reward -
self.values[self.last_state])
```

This is the same TD update but with γ·V(s') = 0, since we're at a terminal state:
`TD Error = r + 0 - V(s) = r - V(s)`

At a terminal state, the only 'future' reward is the immediate reward r. So the update drives V(s) toward just r.

4.5 agent_cleanup() and agent_message()

agent_cleanup(): resets last_state = None between episodes
agent_message('get_values'): returns self.values — the current value function estimates

PART 5: The Three Policies Evaluated

5.1 The Optimal Policy

The optimal policy takes the SHORTEST path just above the cliff:

- State 36 (START): Always go UP → goes to state 24
- States 24–34 (row 2): Always go RIGHT → moves along second-to-bottom row

- State 35: Always go DOWN → reaches goal at state 47

This policy is deterministic (100% probability for one action at each state):

```
policy[36] = [1., 0., 0., 0.]  # UP with probability 1
for i in range(24, 35): policy[i] = [0., 0., 0., 1.]  # RIGHT
policy[35] = [0., 0., 1., 0.]  # DOWN
```

This policy minimizes the number of steps (shortest path = most reward), but risks falling off the cliff if actions were stochastic. In this deterministic environment, it's perfectly safe.

5.2 The Safe Policy

The safe policy takes the long way around, far from the cliff:

- States 12, 24, 36: Always go UP → moves up one row
- States 0–10 (top row): Always go RIGHT → traverses the top
- States 11, 23, 35: Always go DOWN → descends on right side

```
for i in [12, 24, 36]: policy[i] = [1., 0., 0., 0.]  # UP
for i in range(0, 11): policy[i] = [0., 0., 0., 1.]  # RIGHT
for i in [11, 23, 35]: policy[i] = [0., 0., 1., 0.]  # DOWN
```

This policy takes many more steps (longer path) but completely avoids the cliff. In a stochastic environment, this would be safer.

The safe policy has lower expected reward (more -1 penalties per episode) but zero risk of the -100 cliff penalty.

5.3 The Near-Optimal Stochastic Policy

This policy mostly follows the optimal path but with some randomness (10% chance of non-optimal action):

```
policy[36] = [0.9, 0.033, 0.033, 0.034]  # 90% UP
for i in range(24, 35): policy[i] = [0.033, 0.033, 0.034, 0.9]  # 90% RIGHT
policy[35] = [0.033, 0.033, 0.9, 0.034]  # 90% DOWN
```

Due to randomness, the agent occasionally falls off the cliff, making results vary between runs. That's why this experiment is run multiple times and averaged.

PART 6: Mathematical Deep Dive

6.1 The Bellman Equation (What TD(0) Converges To)

TD(0) converges to the solution of the Bellman Expectation Equation:

```
V(s) = Σ_a π(a|s) Σ_s' P(s'|s,a) [R(s,a,s') + γ V(s')]
```

In words: the value of state s equals the expected immediate reward plus the expected discounted value of the next state, averaged over all possible actions (from policy) and transitions.

TD(0) finds this fixed point without ever explicitly computing the sum — it estimates it from samples.

6.2 Convergence Guarantees

TD(0) is guaranteed to converge to the true value function V_π if:

- The policy π is fixed (not being improved — pure prediction)
- Step size α satisfies: $\Sigma\alpha = \infty$ (large enough to overcome initial values)
- Step size α satisfies: $\Sigma\alpha^2 < \infty$ (small enough to converge)
- All states are visited infinitely often

In practice, a constant α (like 0.01) works well for many problems, though it doesn't formally converge — it keeps adapting.

6.3 TD vs Monte Carlo vs Dynamic Programming

Three main approaches to policy evaluation:

Feature	Dynamic Programming	Monte Carlo	TD(0)
Needs model?	YES	NO	NO
Bootstraps?	YES	NO	YES
Online update?	YES	NO (end of episode)	YES
Variance	Zero	High	Low
Bias	Zero	Zero	Some (uses estimates)

6.4 The Discount Factor γ (gamma)

In this lab, $\gamma = 1.0$ (no discounting). This means all future rewards are equally valuable as immediate rewards.

- $\gamma = 1.0$: agent cares equally about all future rewards (used when episodes always terminate)
- $\gamma = 0.9$: agent values rewards 10 steps away at $0.9^{10} \approx 0.35$ of immediate reward
- $\gamma = 0$: agent only cares about immediate reward (greedy/myopic)

With $\gamma = 1$ and the cliff walk, V(start) should be approximately -13 for the optimal policy (path length ~13) or much more negative for the safe policy (longer path) or stochastic policy (risk of cliff).

6.5 The Step Size α (alpha)

In this lab, α = 0.01. This controls how fast the agent updates its value estimates.

- Large α (e.g., 0.9): learns quickly but may oscillate and never truly converge
- Small α (e.g., 0.001): converges stably but requires many more episodes
- α = 0.01: a reasonable balance for this problem (5000 episodes)

Each update moves V(s) only 1% of the way toward the new target. This smooths out noise from random experience.

PART 7: The RL-Glue Framework

What is RL-Glue?

RL-Glue is a software framework that connects an agent and environment through a standardized interface. It manages the interaction loop so you don't have to write boilerplate.

The Interaction Loop

```
rl_glue.rl_init(agent_info, env_info)
  → calls env.env_init() and agent.agent_init()

rl_glue.rl_episode(0)  # 0 = no step limit
  1. state = env.env_start()
  2. action = agent.agent_start(state)
  3. Loop:
       reward, state, terminal = env.env_step(action)
       if terminal: agent.agent_end(reward); break
       action = agent.agent_step(reward, state)
```

The framework separates concerns: the environment doesn't know about the agent's learning algorithm, and the agent doesn't know about the environment's dynamics.

The Manager Class

The Manager class handles visualization and evaluation. It:

- Tracks value function estimates over time
- Computes Root Mean Squared Value Error (RMSVE) against true values
- Plots learned value functions as heatmaps
- Shows per-state value errors
- Plots RMSVE convergence curves

PART 8: Understanding the Experimental Results

8.1 What the Value Function Tells You

The value function V(s) represents the expected total (discounted) reward starting from state s and following the policy forever.

- More negative value = worse state (agent expects to accumulate more penalties from here)
- Less negative value = better state (closer to goal, fewer steps needed)
- Goal state V(goal) ≈ 0 (episode ends immediately, minimal future reward)
- States near cliff with stochastic policy: very negative (risk of -100 penalty)

8.2 Optimal Policy Results

Expected observations:

- Values most negative at start (far from goal, ~13 steps away = ~-13)
- Values increase (less negative) as agent gets closer to goal
- RMSVE curve decreases smoothly toward 0 as estimates improve
- Converges quickly because path is deterministic and short

8.3 Safe Policy Results

Expected observations:

- All values more negative than optimal policy (longer path = more penalties)
- Top row has decent values (on the path)
- Bottom rows below the path have poor values (agent must travel far)
- RMSVE also converges to 0 but may take longer (more varied experience needed)

8.4 Stochastic Policy Results

Expected observations:

- Very negative values near the cliff (10% chance of catastrophic -100)
- High variance between runs (randomness causes different trajectories)
- RMSVE may not converge to 0 with just one run — needs averaging
- Demonstrates why averaging over multiple runs is important

8.5 RMSVE — Root Mean Squared Value Error

Formula: RMSVE = sqrt(mean((V_estimated(s) - V_true(s))^2))

- Measures how far our estimates are from the true value function
- Zero means perfect estimation
- Decreasing over episodes means the agent is learning
- Requires knowing true values (provided in .npy files in this lab)

PART 9: Common Exam Questions and Answers

Q1: What is the difference between prediction and control in RL?

Prediction (policy evaluation): given a fixed policy, estimate V(s) for all states. No policy improvement.
Control: find the OPTIMAL policy. Involves both evaluation and improvement. TD(0) is a prediction algorithm.

Q2: Why does TD(0) update after every step, unlike Monte Carlo?

Monte Carlo must wait until the episode ends to compute the actual return $G_t = R_1 + \gamma R_2 + \gamma^2 R_3 + ...$ Because future rewards are unknown during the episode.
TD(0) uses the ESTIMATE V(s') instead of the actual future return. This bootstrapping allows immediate updates without waiting for episode completion.

Q3: What happens if α is too large or too small?

Too large ($\alpha \approx 1$): estimates swing wildly, may not converge, high variance.
Too small ($\alpha \approx 0.0001$): converges but needs millions of episodes, very slow.
Just right ($\alpha \approx 0.01$–0.1): good balance of speed and stability.

Q4: Why is γ = 1.0 used in the cliff walking experiment?

Because the task is episodic (always terminates at the goal). With guaranteed termination, $\gamma = 1$ is fine — infinite future rewards are impossible. If the task could run forever, $\gamma < 1$ would be needed to ensure convergence.

Q5: What does the cliff walking environment test?

It tests the agent's ability to balance risk vs. reward: the optimal path (along the cliff) is shorter but riskier; the safe path is longer but avoids the cliff. Different policies make different trade-offs, and TD(0) learns the value of each policy from experience.

Q6: Why does the agent reset to start after falling off the cliff?

The problem is designed this way: falling off the cliff is a severe but non-terminal event. The agent must learn to AVOID the cliff through experience (the -100 penalty signal), not have the episode end. This makes the problem harder and more interesting.

Q7: What is the role of the policy in TD(0)?

The policy π is FIXED throughout learning in TD(0). The agent samples actions according to π at each state. The policy determines WHICH states get visited and HOW OFTEN. TD(0) evaluates this fixed policy — it does NOT improve it.

Q8: How does bootstrapping create bias in TD(0)?

V(s') is an estimate, not the true value. So the target r + γ·V(s') is only as accurate as our current estimate V(s'). Early in training, V(s') is far from true, introducing bias. Over time, as estimates improve, the bias decreases. Monte Carlo has no bias because it uses actual returns.

Q9: If the state() function is wrong, what breaks?

Everything. The state index is used to look up values in self.values[state] and to look up policy probabilities in self.policy[state]. A wrong state mapping means wrong value updates and wrong action selection.

Q10: What does env_start() return and why?

It returns the initial state (as an integer). The agent needs this to select its first action. Without the initial state, the agent has no information to start with.

PART 10: Key Formulas Summary

Memorize These

State conversion: state = row × grid_width + col

TD(0) update: $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$

TD Error: $\delta = r + \gamma V(s') - V(s)$

Terminal update: $V(s) \leftarrow V(s) + \alpha[r - V(s)]$ [because $\gamma V(\text{terminal}) = 0$]

RMSVE: $\text{sqrt}(\text{mean}((V_{est}(s) - V_{true}(s))^2))$

Bellman equation: $V_\pi(s) = \Sigma_a \pi(a|s) [R(s,a) + \gamma \Sigma_{s'} P(s'|s,a) V_\pi(s')]$

Parameter Values in This Lab

- Grid: 4 rows × 12 columns = 48 states
- Actions: 4 (UP=0, LEFT=1, DOWN=2, RIGHT=3)

- Discount $\gamma = 1.0$
- Step size $\alpha = 0.01$
- Episodes = 5000
- Plotting frequency = every 500 episodes
- Normal step reward = -1
- Cliff penalty = -100
- Goal reward = -1 (then terminal)
- Start state = 36 = (3,0)
- Goal state = 47 = (3,11)
- Cliff states = 37 through 46 = (3,1) through (3,10)

End of Study Guide — Assignment 6-1: Policy Evaluation with TD(0)